

This requires us to understand and use low-level thread-creation APIs such as *pthread_create* to create the thread, and *p_thread_join* to join the auxiliary thread with the main thread. However, with the support for threading in C++11, developers need not concern themselves with these low-level details. Here is a code snippet that demonstrates the 'Hello World' code in C++11:

```
#include <iostream>
#include <thread>
//Function called from auxiliary thread
void print_hello(void) {
    std::cout << "Hello, World!" << std::endl;
}
int main()
{
    std::cout << "Hello, World from Main Thread" <<
std::endl;
    //Create the auxiliary thread
    std::thread thread1(print_hello);
    //join the auxiliary thread with main thread
    thread1.join();
    return 0;
}
```

In the above example, we used *std::thread* to create a new thread, which runs the task '*print_hello*'. All the low-level details of using *pthread* interfaces are now hidden for the developer. A rough skeleton of the thread class in C++11 is shown below:

```
class thread {
public:
    class id;
    template <class F, class ... Args> thread (F&& f,
Args&&... args);
    bool joinable() const;
    void join();
    void detach();
    id getid const();
    ....
}
```

In the above example, we did not pass any arguments to the function '*print_hello*', with which we invoked the auxiliary thread. Let us assume that we decided to pass in an integer variable to '*print_hello*'. Here is how the modified code looks:

```
#include <iostream>
#include <thread>
//Function called from auxiliary thread
```

```
void print_hello(int j) {
    std::cout << "Hello, World!" << j << std::endl;
}
int main()
{
    int i = 2; //signifying that it is the second thread
    std::cout << "Hello, World from Main Thread" <<
std::endl;
    //Create the auxiliary thread
    std::thread thread1(print_hello, i);
    //join the auxiliary thread with main thread
    thread1.join();
    return 0;
}
```

Still now, we have created only one thread, apart from the main thread (recall that every program, by default, starts off with a single thread of execution). It is quite straightforward to modify the above piece of code to create a group of threads and invoke them with different functions to execute. I leave that as an exercise for our readers.

Remember, threads in a process share the data associated with that process. Hence, shared data needs to be protected against concurrent accesses from threads. This is typically achieved using explicit synchronisation in the form of locks. For instance, if we have a shared integer counter variable that can be incremented by multiple threads concurrently, we would associate a '*pthread_mutex*' to protect the shared counter from concurrent accesses by threads, as shown below:

```
pthread_mutex_t my_mutex;
int global_counter;
int increment(void)
{
    pthread_mutex_lock(&my_mutex);
    global_counter++;
    pthread_mutex_unlock(&my_mutex);
}
```

The C++11 standard also provides support for synchronisation between threads using mutexes. For instance, we can implement the locked counter example as follows:

```
std::mutex my_mutex;
int increment(void)
{
    my_mutex.lock();
    global_counter++;
    my_mutex.unlock();
}
```